



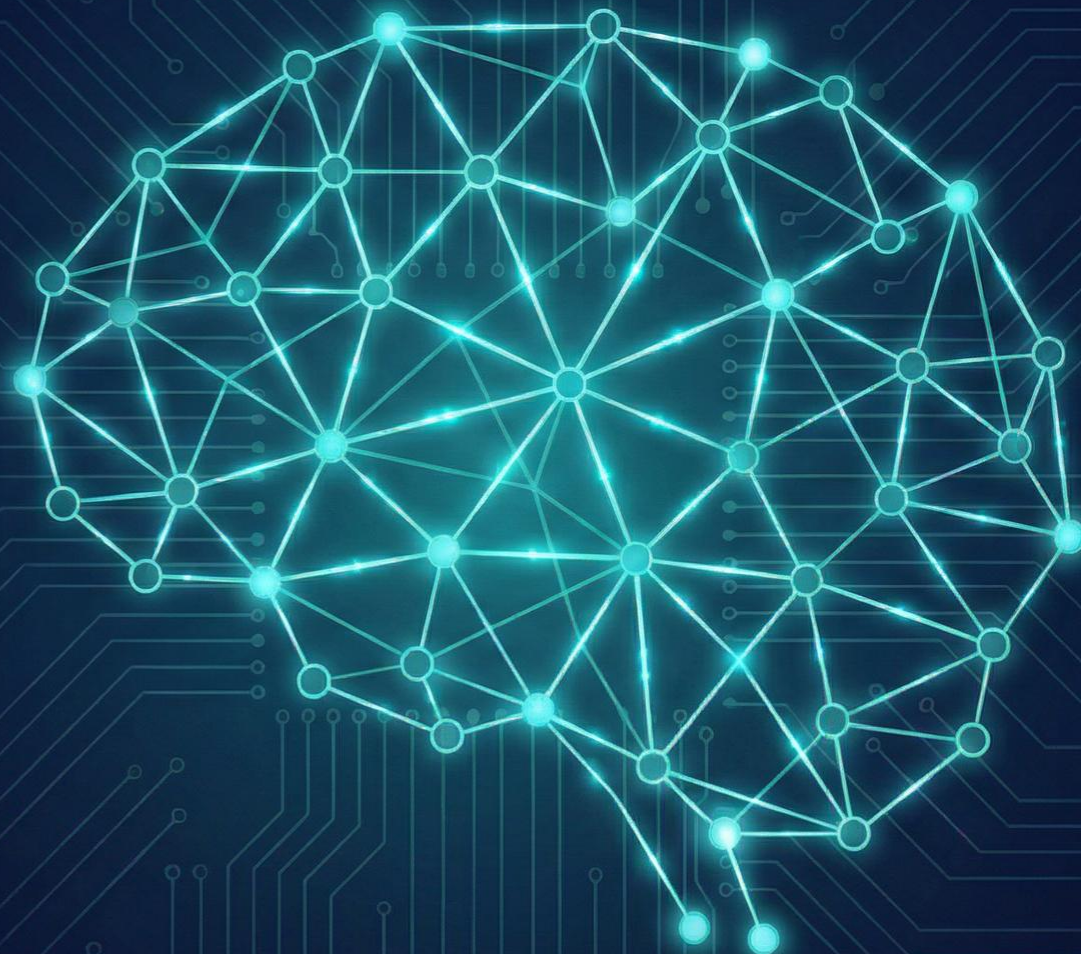
University of
Nottingham

UK | CHINA | MALAYSIA

Fundamentals of Artificial Intelligence

Session 3: Uninformed Search |
AE1FAI / COMP1037

Dr. Qian Zhang | University of Nottingham
Ningbo China | Spring



Session 3 Overview

1. Recap: Problem Formulation & Search
2. Breadth-First Search (BFS): Go wide before deep
3. Depth-First Search (DFS): Go deep before wide
4. Uniform-Cost Search (UCS): The cheapest path first
5. Evaluation & Summary: Comparing the strategies



By the end of this session, you will be able to trace, implement, and evaluate all three algorithms.

Four Components



Initial State: Where the agent starts



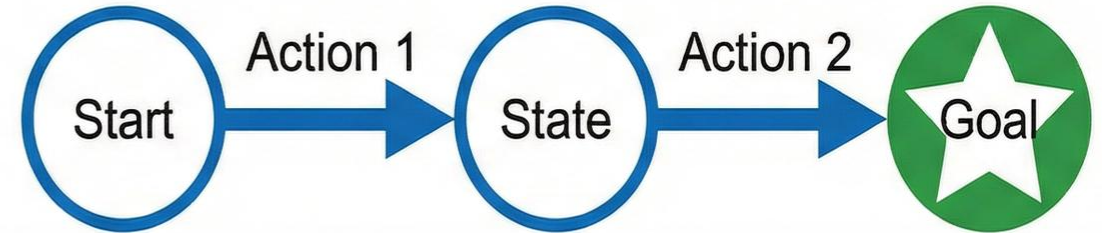
Actions: What the agent can do



Goal Test: How to know if goal is reached

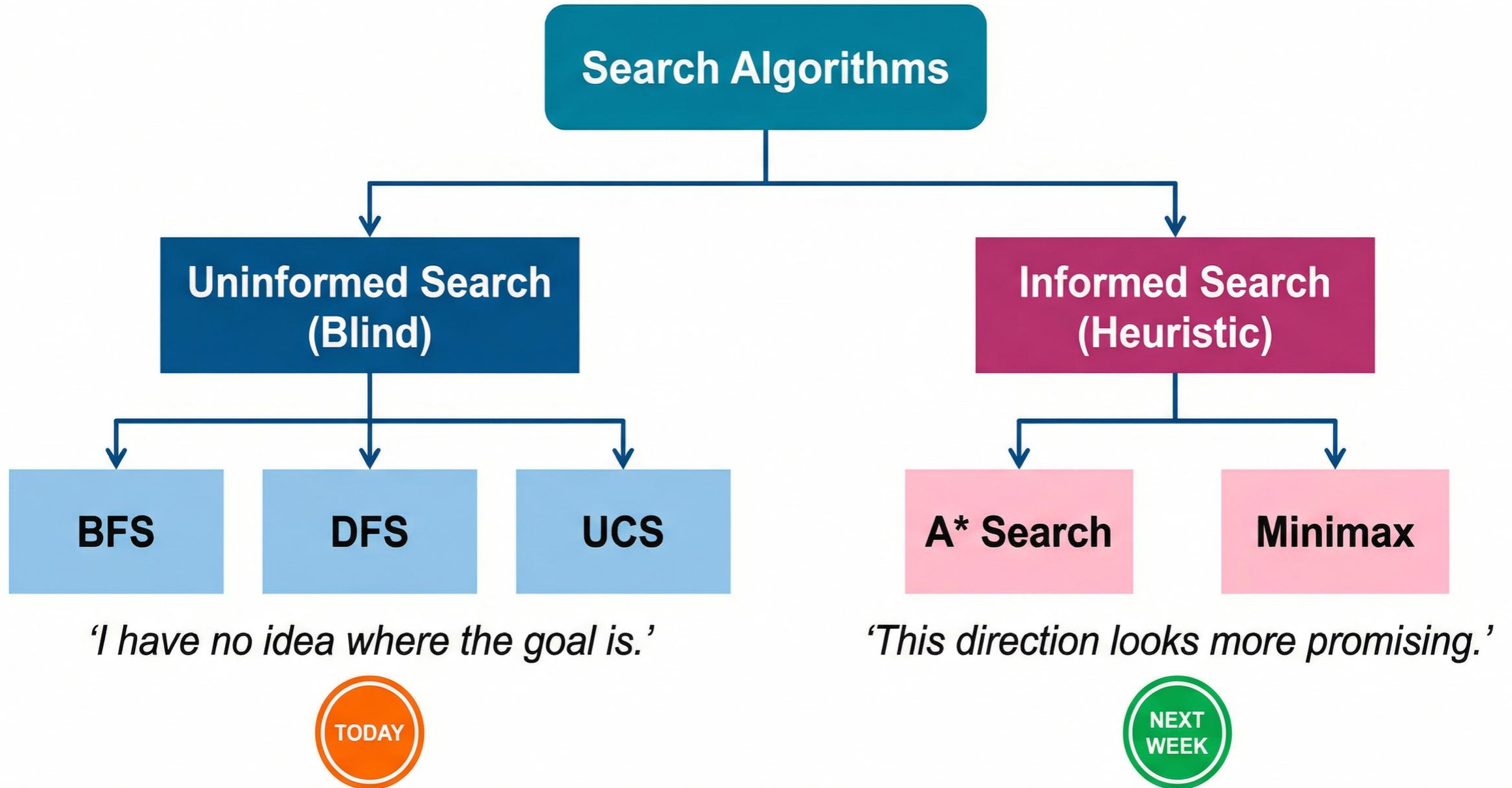


Path Cost: Cost of a sequence of actions





Two Families of Search Algorithms





Four Evaluation Criteria

Criteria	Question
#1 Completeness	Does it always find a solution if one exists?
#2 Optimality	Does it always find the best (least-cost) solution?
#3 Time Complexity	How long does it take? (Nodes generated)
#4 Space Complexity	How much memory does it need? (Nodes stored)





Two Fundamental Actions

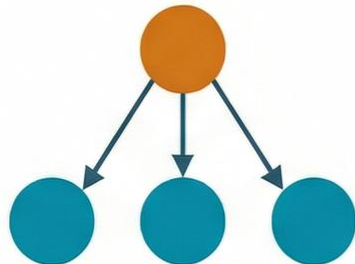


Action 1: Expand

Ask a node for its children

EXPAND(node, problem) → returns a list of child nodes reachable from this node

```
def expand(problem, node, get_actions_fn, action_cost_fn):
    children = []
    for action_name, next_state in get_actions_fn(problem, node['state']):
        cost = action_cost_fn(problem, node['state'], action_name)
        children.append(make_node(next_state, node, action_name,
                                   node['cost'] + cost, node['depth'] + 1))
    return children
```



Every search algorithm uses EXPAND to generate children and TEST to check for goal

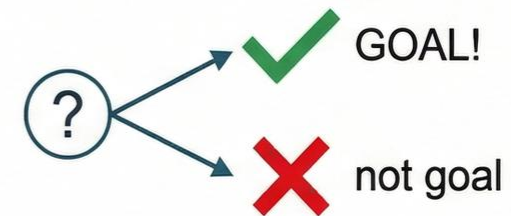


Action 2: Test (Goal Test)

Test a node to see whether it is a goal

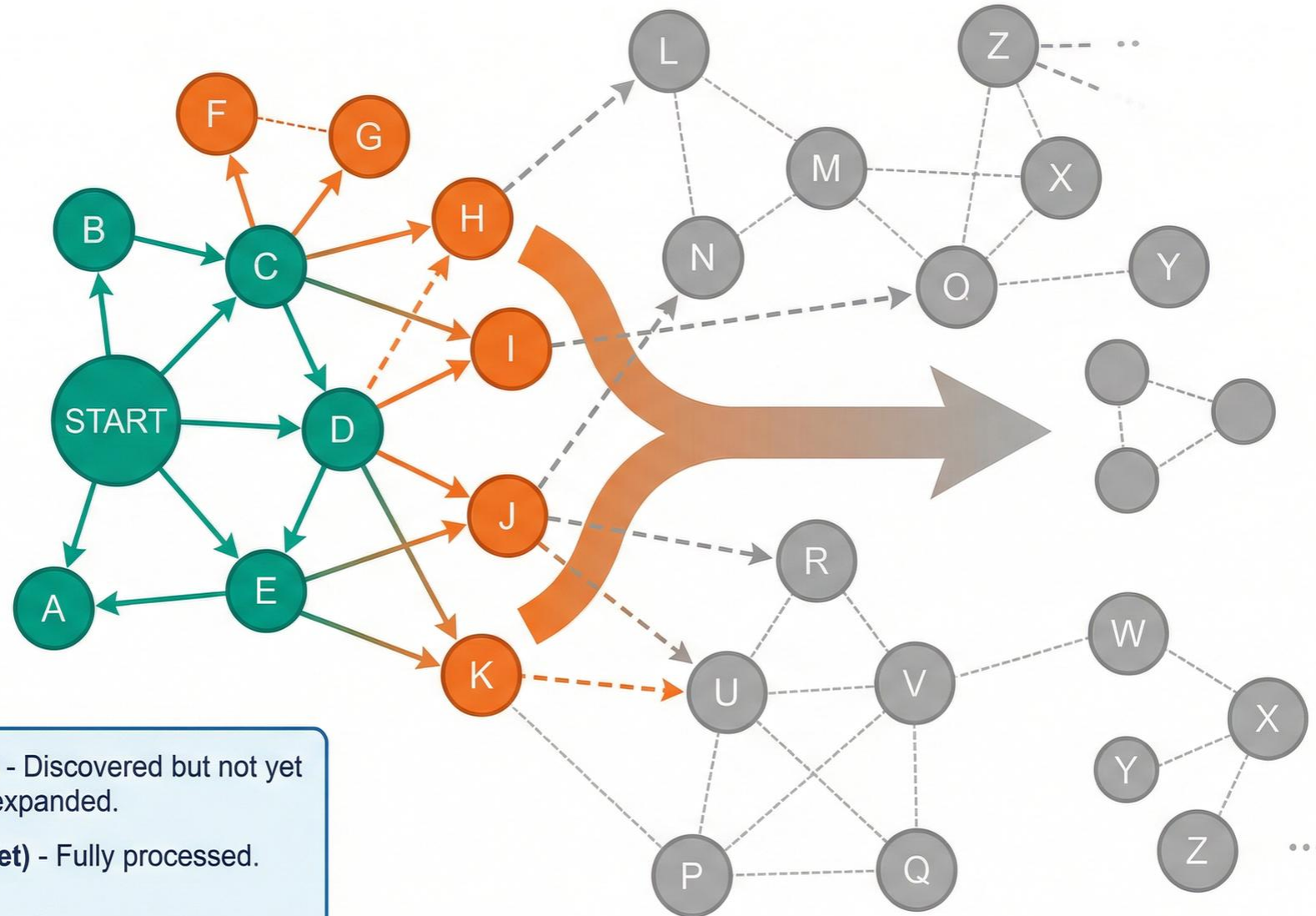
GOAL-TEST(problem, state) → returns True if state is the goal

```
def is_goal(problem, state):
    return state == problem['goal']
```





Keeping Track: Frontier, Explored, Undiscovered



FRONTIER (Open Set) - Discovered but not yet processed. Next to be expanded.



EXPLORED (Closed Set) - Fully processed. Already expanded.



UNDISCOVERED - Not yet seen by the search.



The General Search Algorithm

All blind search algorithms share the same skeleton:

```
function GENERAL-SEARCH(problem, QUEUING-FN):  
  nodes = MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[problem]))  
  loop do:  
    if nodes is EMPTY then return FAILURE  
    node = REMOVE-FRONT(nodes)  
    if GOAL-TEST(problem, STATE(node)) then return node  
    nodes = QUEUING-FN(nodes, EXPAND(node, problem))
```

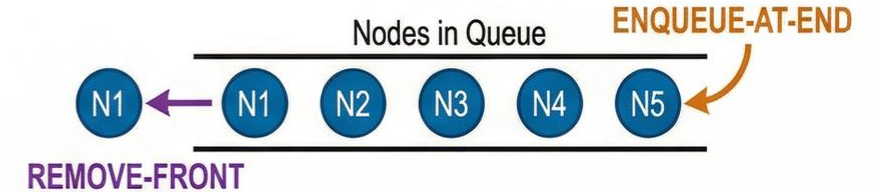
BFS: QUEUING-FN = ENQUEUE-AT-END (FIFO Queue)

DFS: QUEUING-FN = ENQUEUE-AT-FRONT (LIFO Stack)

UCS: QUEUING-FN = INSERT-BY-PATH-COST (Priority Queue)

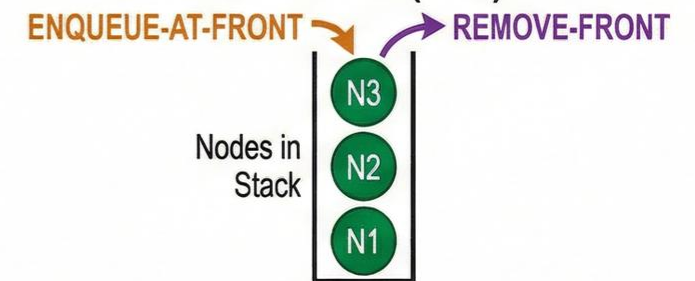
Queue Types (QUEUING-FN):

FIFO Queue (BFS)



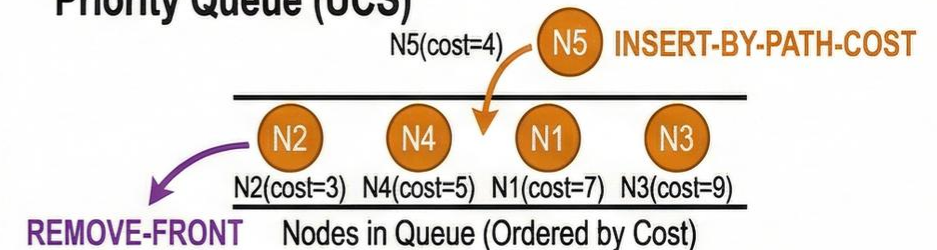
First-In, First-Out. Explores breadth-wise.

LIFO Stack (DFS)



Last-In, First-Out. Explores depth-wise.

Priority Queue (UCS)



Lowest Cost First. Explores by path cost.

The General Search Algorithm: Code

PSEUDOCODE (from AIMA)

```
function GENERAL-SEARCH(problem, QUEUING-FN):
    frontier = MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[p]))
    loop do
        if frontier is EMPTY then return FAILURE
        node = REMOVE-FRONT(frontier)
        if GOAL-TEST[p](STATE(node)) then return node
        frontier = QUEUING-FN(frontier,
                               EXPAND(node, ACTIONS[p]))
    end
```

PYTHON IMPLEMENTATION

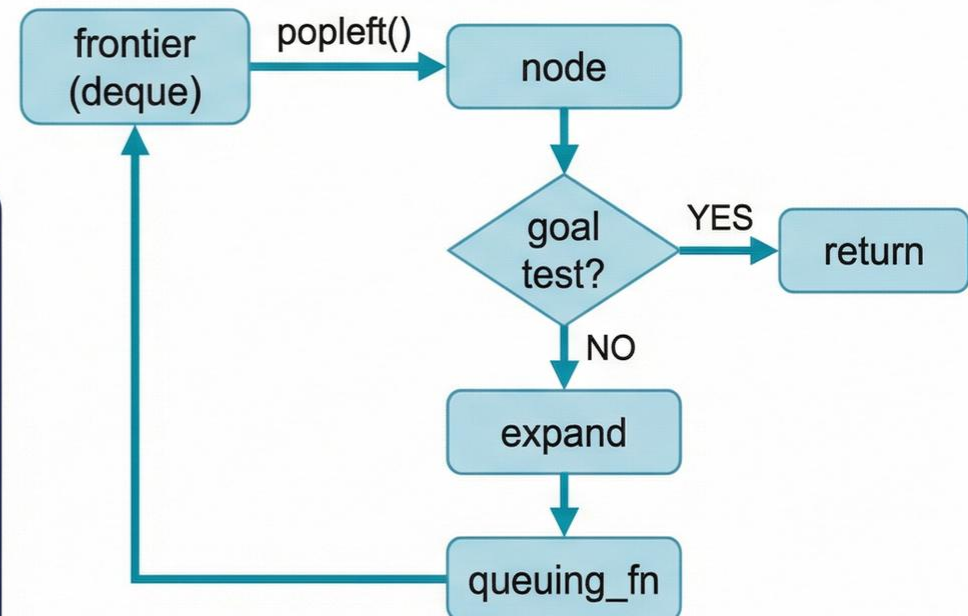
```
def general_search(problem, queuing_fn,
                   get_actions_fn, action_cost_fn):
    frontier = deque([make_node(problem['initial'])]) # Make-Queue
    while True:                                     # Loop do
        if not frontier: return None                # Empty → failure
        node = frontier.popleft()                   # Remove-Front
        if is_goal(problem, node['state']):          # Goal-Test
            return node
        queuing_fn(frontier,                          # QUEUING-FN
                    expand(problem, node,
                            get_actions_fn, action_cost_fn))
```

Key Insight

BFS: queuing_fn = enqueue_at_end (FIFO Queue)

DFS: queuing_fn = enqueue_at_front (LIFO Stack)

UCS: queuing_fn = enqueue_by_cost (Priority Queue)

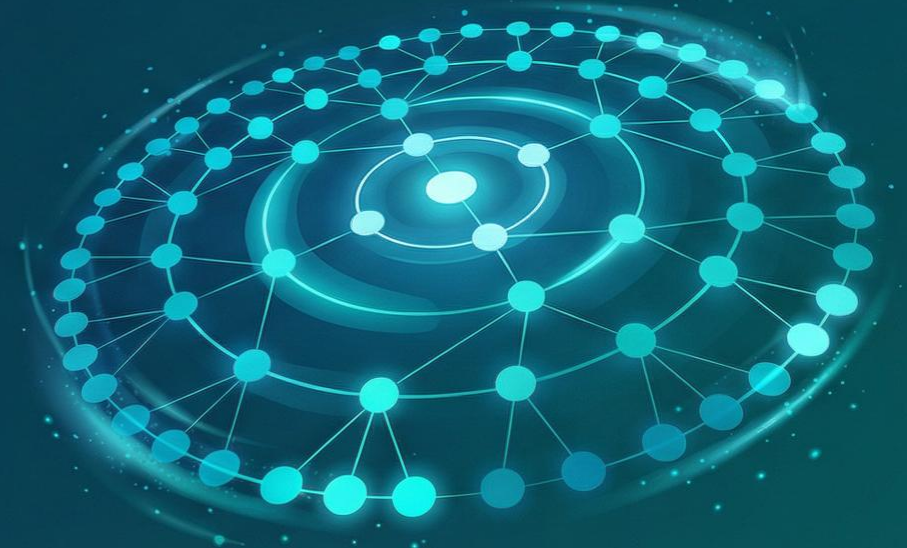




University of
Nottingham
UK | CHINA | MALAYSIA

Breadth-First Search (BFS)

Go wide before you go deep.



Breadth-First Search: The Core Idea

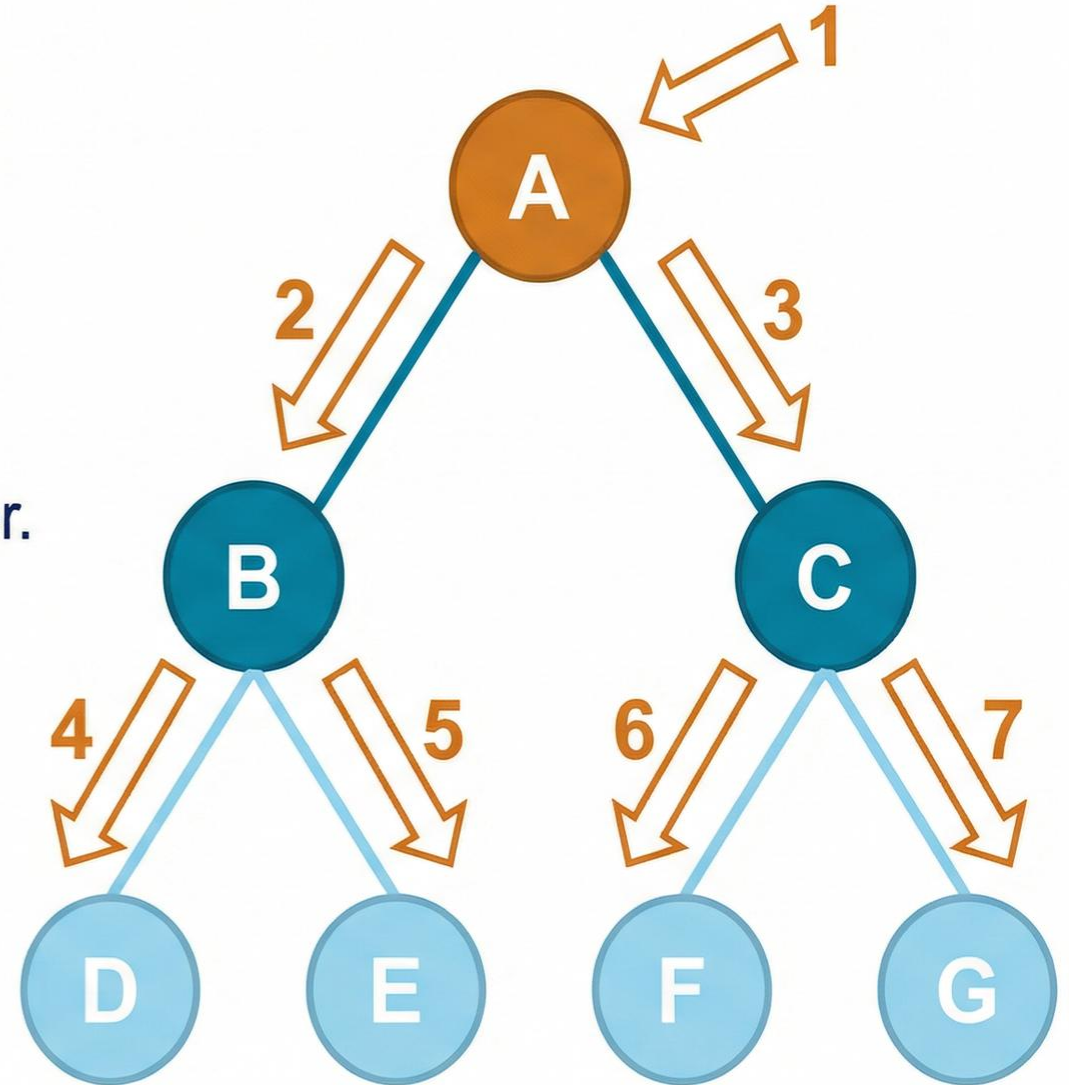
KEY IDEA: Expand the SHALLOWEST unexpanded node first.

Implementation: Use a FIFO Queue for the frontier.

- New nodes added to the BACK of the queue
- Next node to expand taken from the FRONT

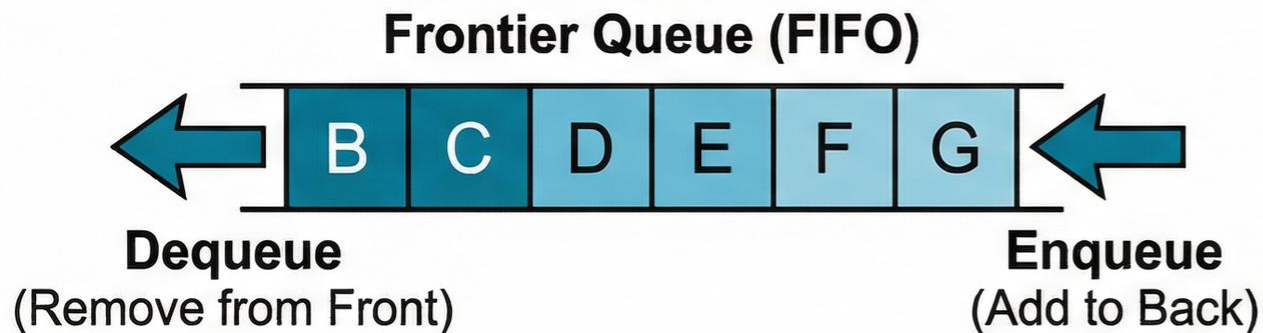
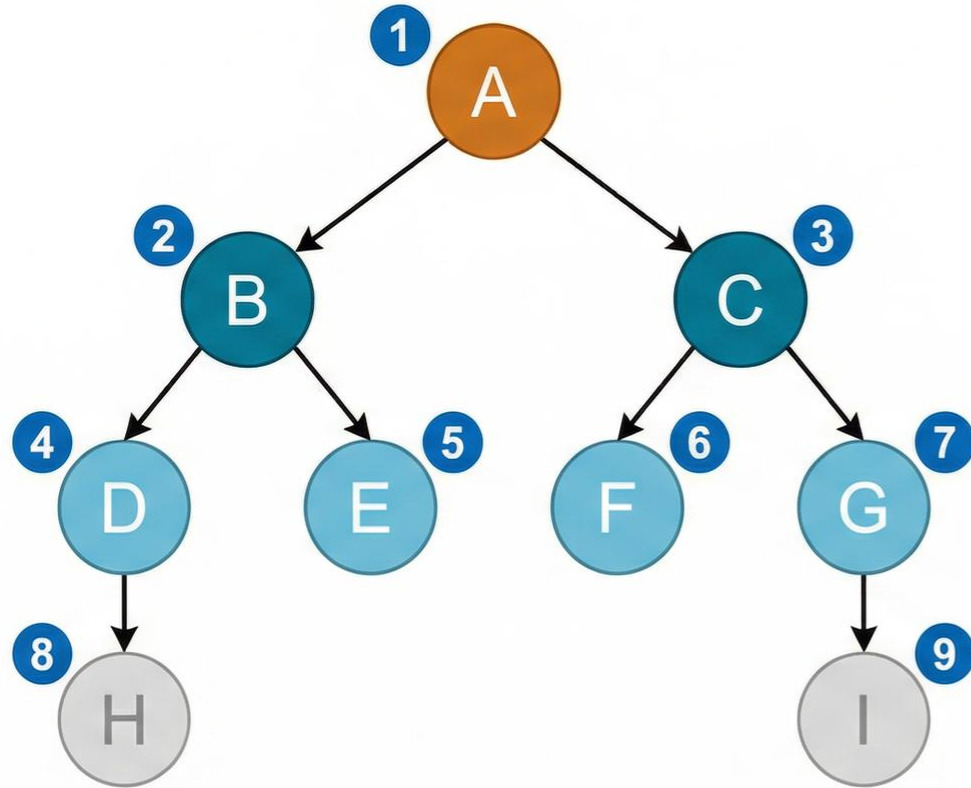
Expansion Order (branching factor $b=2$):

- Level 0: Root (A)
- Level 1: Children of A [B, C]
- Level 2: Children of B and C [D, E, F, G]





BFS: Step-by-Step Example



INTERACTIVE DEMO

BFS Tree Traversal

Open [bfs_demo.html](#) in browser

● **Step: 3 / 9**

Frontier Queue: [C | D | E | F | G]



← Prev

Next →

Reset

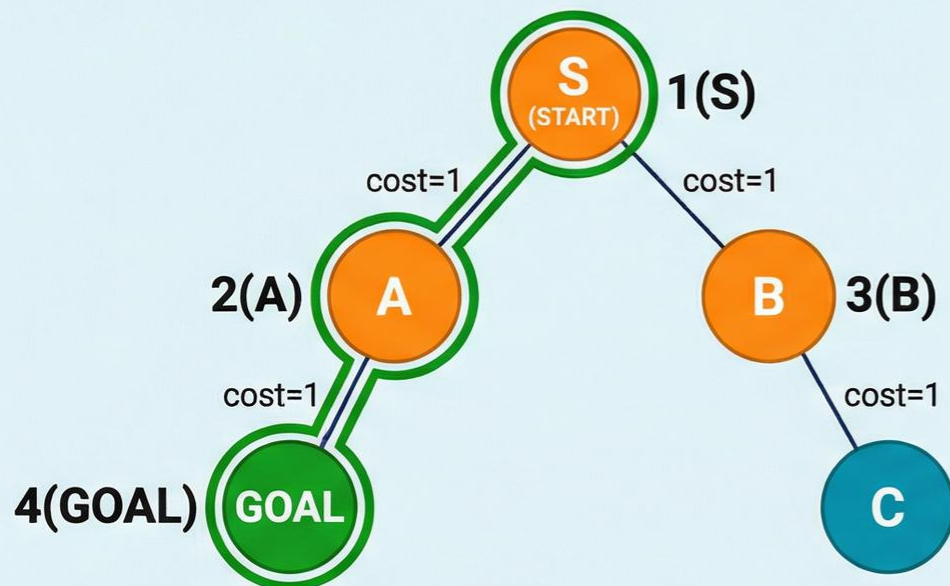
Watch how nodes are added to BACK
and removed from FRONT of queue



BFS: Complete and Optimal

BFS is COMPLETE

Always finds a solution if one exists



BFS finds GOAL at depth=2, cost=2 ✓ OPTIMAL

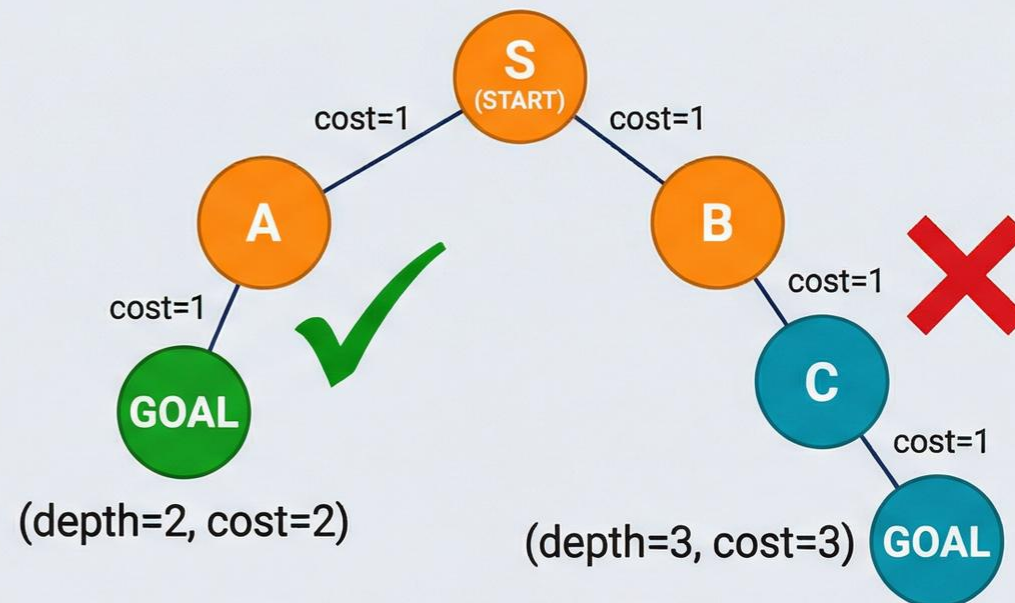
BFS Queue (FIFO - First-In-First-Out)



VS

BFS is OPTIMAL (equal costs)

Finds the shallowest = cheapest solution



(depth=2, cost=2)

(depth=3, cost=3)

Path	Depth	Cost	BFS picks?
S→A→GOAL	2	2	Yes
S→B→C→GOAL	3	3	No (deeper)

- ✓ BFS is **COMPLETE**: Always finds shallowest goal |
- ✓ BFS is **OPTIMAL** when all step costs are **EQUAL**

When all costs equal, fewest steps = lowest cost.
BFS guarantees optimal solution!



Evaluating Breadth-First Search

Criteria	BFS Result	Verdict
Completeness	YES — if branching factor b is finite	✓
Optimality	YES — if all step costs are equal	✓
Time Complexity	$O(b^d)$ — EXPONENTIAL	⚠
Space Complexity	$O(b^d)$ — EXPONENTIAL	⚠

⚠ WARNING: Both time AND space are exponential. This is BFS biggest weakness.

BFS Memory: How Bad is Exponential?

$b = 10$ (branching factor), each level stores ALL nodes

$d=2$

100 nodes

$d=4$

10,000 nodes

$d=6$

1,000,000 nodes

$d=8$

100,000,000 nodes

At $d=8$ with $b=10$: BFS must store **100 MILLION** nodes in memory simultaneously!



BFS: ENQUEUE-AT-END

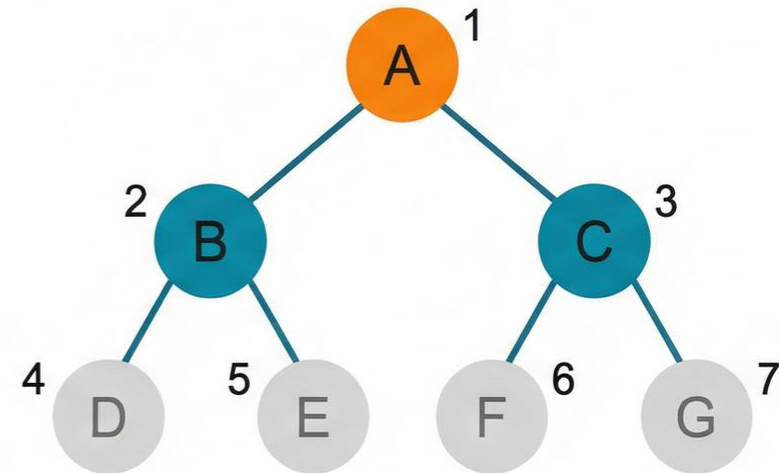
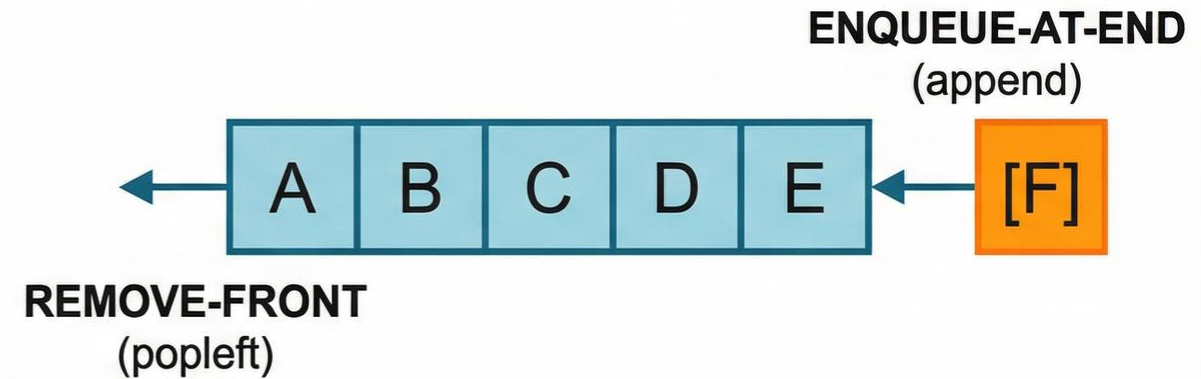
BFS QUEUING FUNCTION

```
def enqueue_at_end(frontier, children):  
    """BFS: ENQUEUE-AT-END  
    New nodes go to the BACK of the queue."""  
    for child in children:  
        frontier.append(child)  # add to BACK
```

RUN BFS

```
# Run BFS on 5x5 grid  
p = make_grid_problem((0,0), (4,4))  
  
print('=== BFS (ENQUEUE-AT-END) ===')  
bfs_result = general_search(  
    p, enqueue_at_end,  
    get_actions, action_cost)  
bfs_path = reconstruct_path(bfs_result)  
print('Path:', bfs_path)
```

How FIFO Queue Works



Explores level by level (shallowest first)



University of
Nottingham
UK | CHINA | MALAYSIA

Depth-First Search (DFS)

Go deep before you go wide.



Depth-First Search: The Core Idea

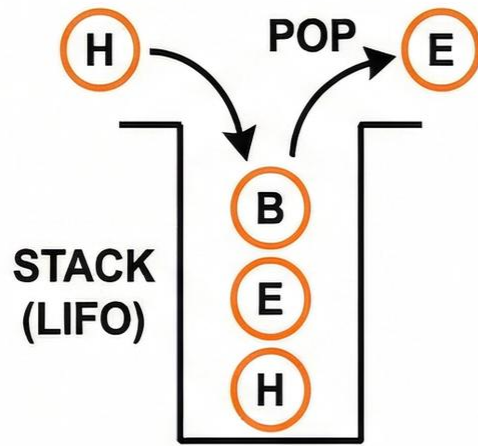
KEY IDEA:

Expand the **DEEPEST** unexpanded node first.

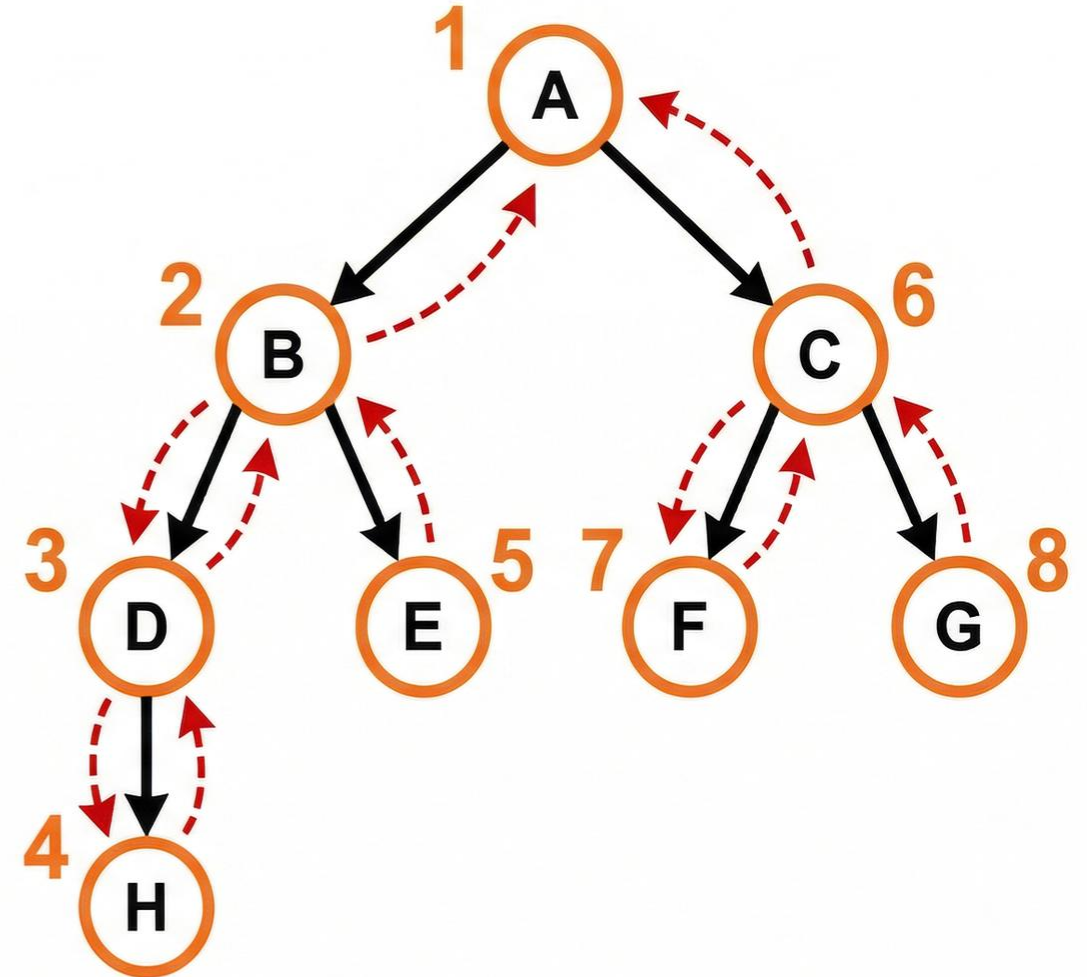
Implementation: Use a LIFO Stack for the frontier.

- New nodes pushed to the TOP of the stack
- Next node to expand popped from the TOP

DFS goes all the way down one path first, then backtracks.



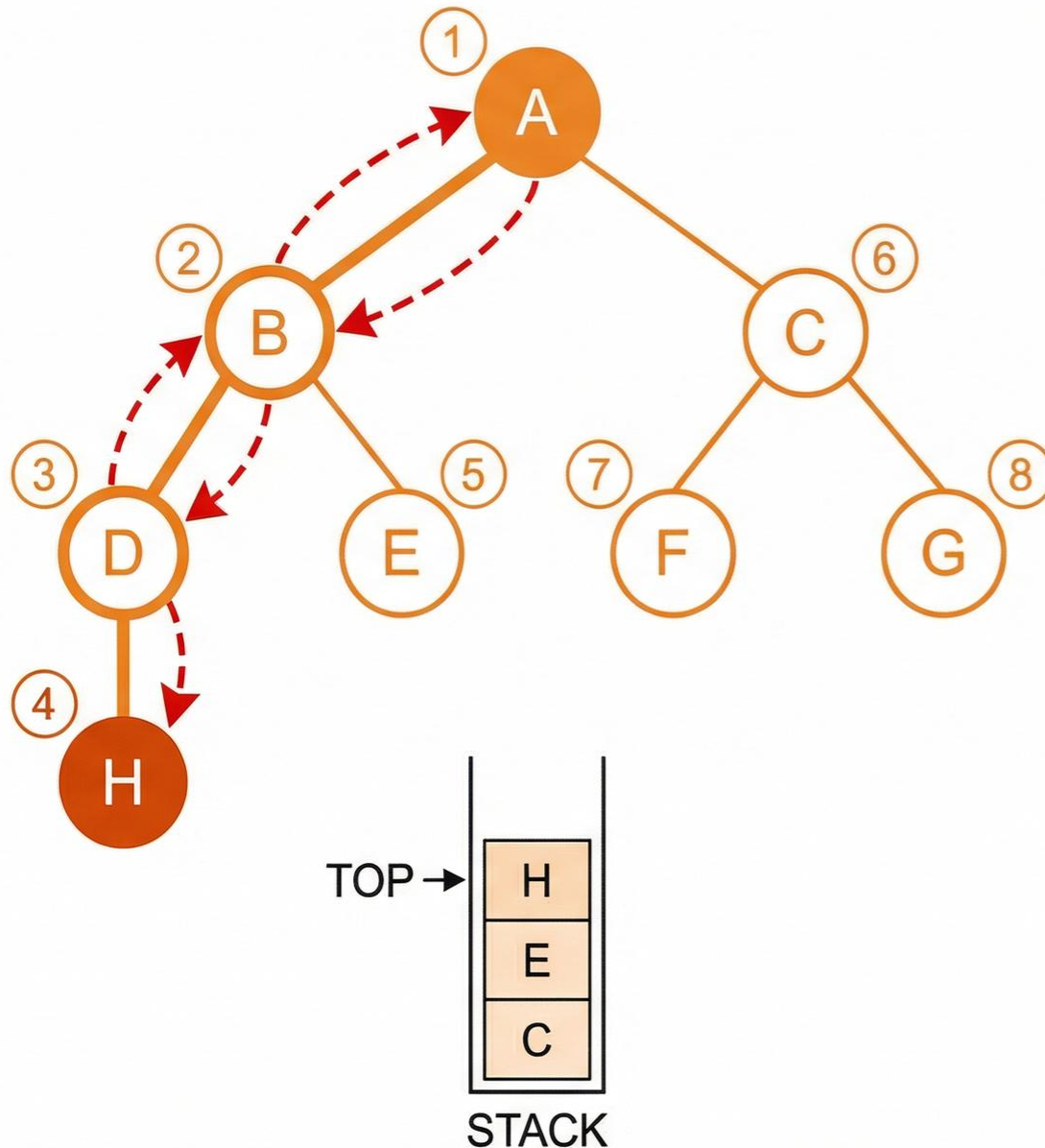
Stack Diagram: Push/Pop Operations



DFS Expansion Order and Backtracking



DFS: Step-by-Step Example



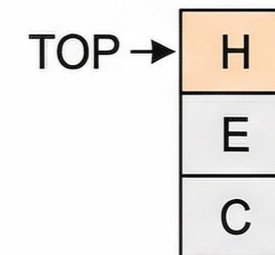
INTERACTIVE DEMO

DFS Tree Traversal

Open [dfs_demo.html](#) in browser

Step: 4 / 8

Current Stack State:



[Prev]

[Next]

[Reset]

Watch how nodes are PUSHED to top
and POPPED from top of stack

DFS: ENQUEUE-AT-FRONT

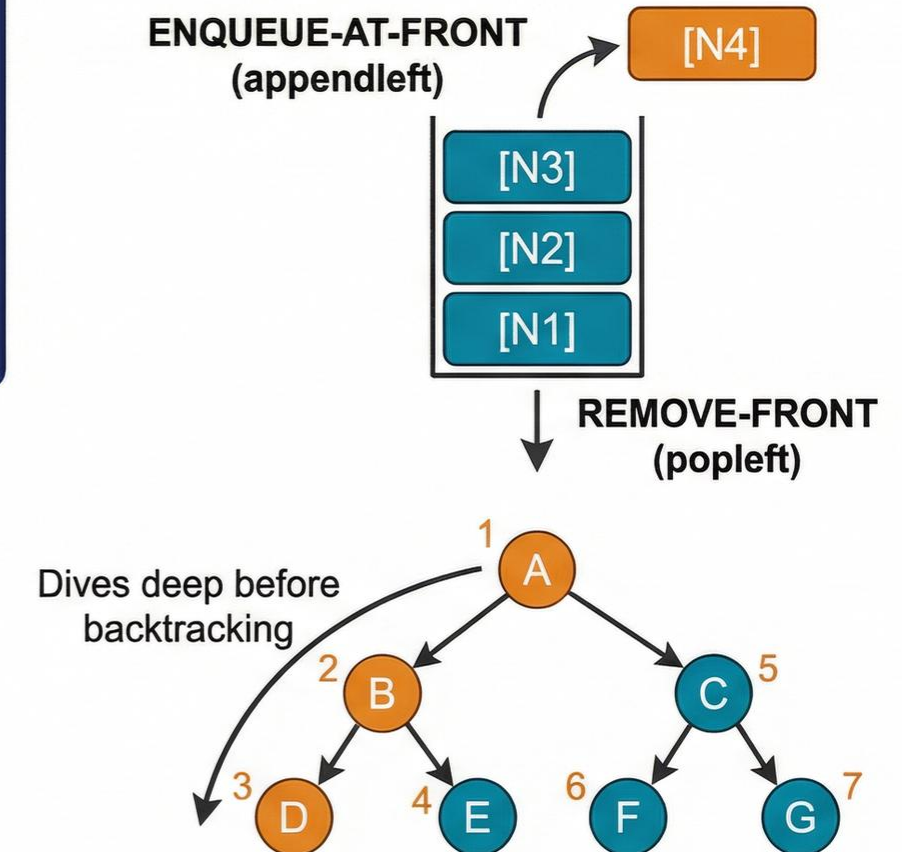
DFS QUEUING FUNCTION

```
def enqueue_at_front(frontier, children):  
    """DFS: ENQUEUE-AT-FRONT  
    New nodes go to the FRONT (LIFO).  
    is_cycle filters out nodes already on path."""  
    for child in reversed(children):  
        if not is_cycle(child):          # skip cycles  
            frontier.appendleft(child)  # add to FRONT
```

RUN DFS

```
# Run DFS  
print('=== DFS (ENQUEUE-AT-FRONT) ===')  
dfs_result = general_search(  
    p, enqueue_at_front,  
    get_actions, action_cost)  
dfs_path = reconstruct_path(dfs_result)  
print('Path:', dfs_path)
```

How LIFO Stack Works

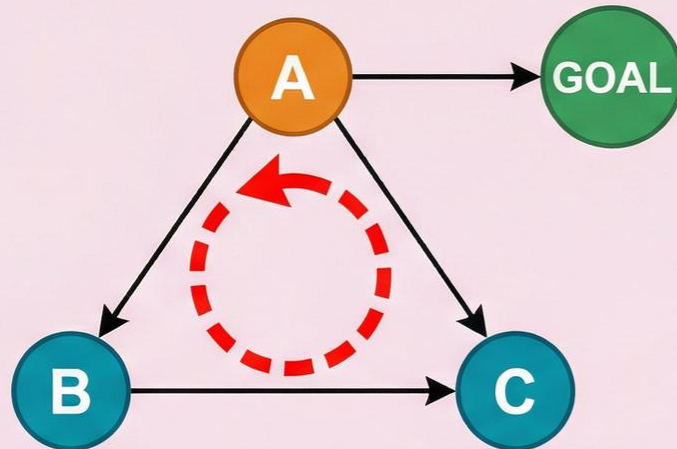


Cycle Prevention: `is_cycle()` checks if state already on current path

DFS: Incomplete and Not Optimal

DFS is INCOMPLETE

Can get stuck in infinite loops forever



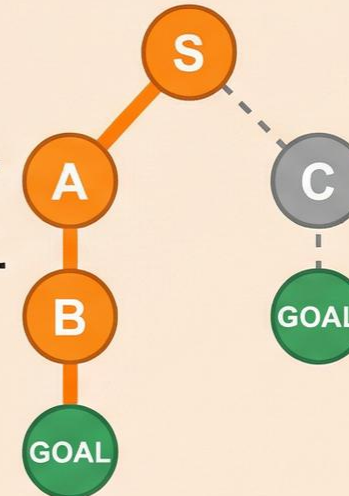
! DFS gets stuck!
 $A \rightarrow B \rightarrow C \rightarrow A \rightarrow B \rightarrow C \dots$
NEVER reaches **GOAL**

DFS is **INCOMPLETE**:
May never find a solution

DFS is NOT OPTIMAL

Finds leftmost path, not shortest/cheapest

DFS finds:
 $S \rightarrow A \rightarrow B \rightarrow \text{GOAL}$,
depth=3, cost=3
✗ NOT OPTIMAL



Optimal:
 $S \rightarrow C \rightarrow \text{GOAL}$,
depth=2, cost=2
✓

Path	Depth	Cost	DFS picks?
$S \rightarrow A \rightarrow B \rightarrow \text{GOAL}$	3	3	Yes (First Found)
$S \rightarrow C \rightarrow \text{GOAL}$	2	2	No (Missed)

DFS picks the **FIRST** path found
(leftmost), not the **best** path!



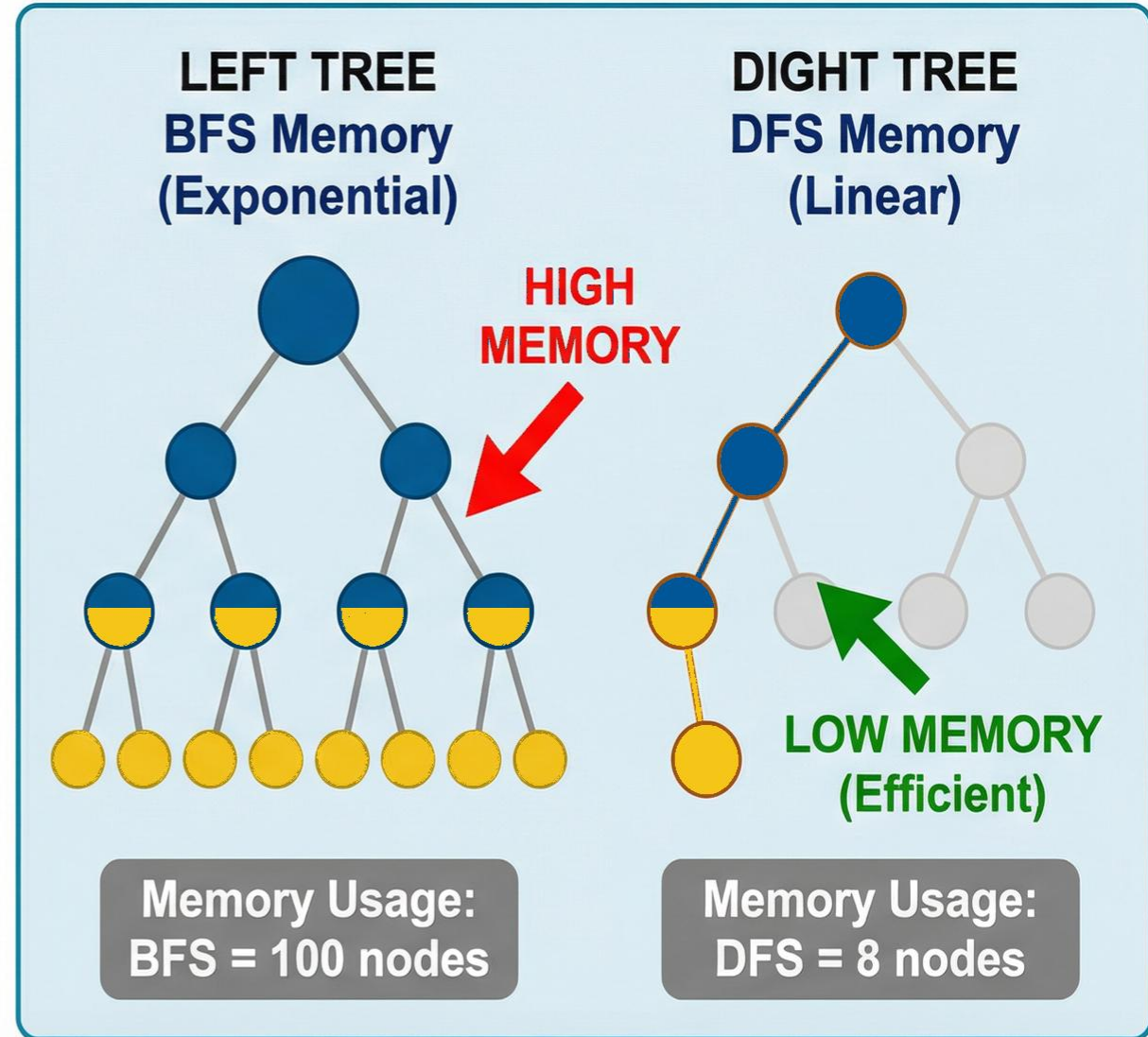
Evaluating Depth-First Search

Evaluation Table

Criteria	DFS Result	Verdict
Completeness	NO - can get stuck in infinite loops	✗
Optimality	NO - finds leftmost solution, not best	✗
Time Complexity	$O(b^m)$ where m = max depth	⚠
Space Complexity	$O(b*m)$ - LINEAR! This is its advantage!	✓★

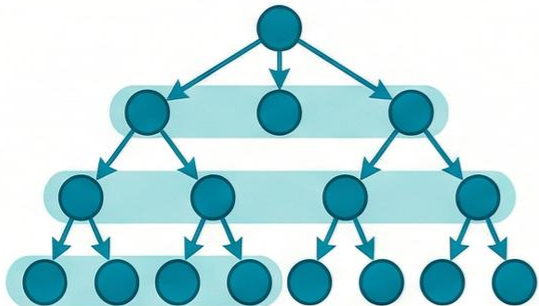
DFS ADVANTAGE: Space complexity is **LINEAR**, not exponential. Only needs to store the current path!

Memory Usage Comparison



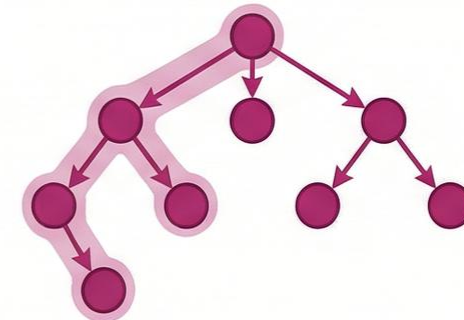
BFS

- ✓ Finds the SHALLOWEST solution
- ✓ COMPLETE and OPTIMAL
- ✓ Use when solution is not deep
- ✗ HUGE memory usage $O(b^d)$



DFS

- ✓ Very LOW memory $O(b*m)$
- ✓ Can find solution fast (if lucky)
- ✗ NOT complete, NOT optimal
- ✗ Can get lost in infinite paths

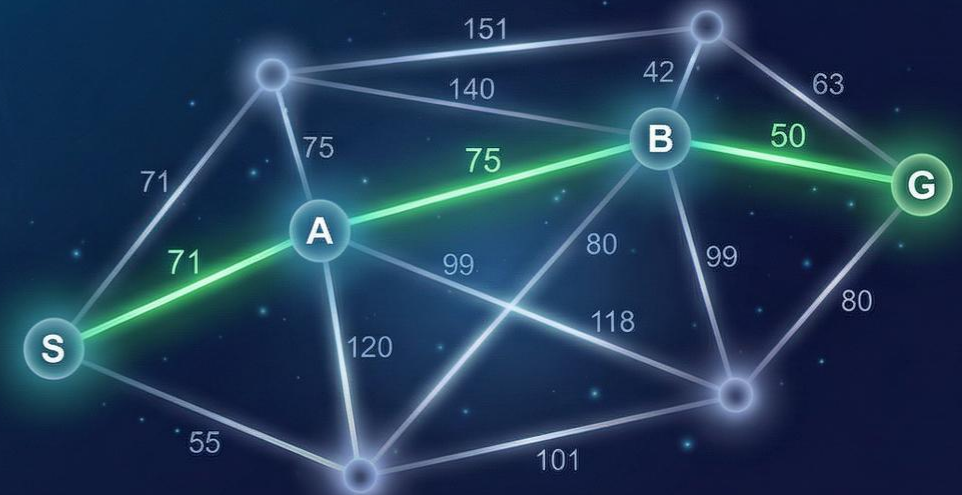


CHOOSE BASED ON YOUR PROBLEM CONSTRAINTS



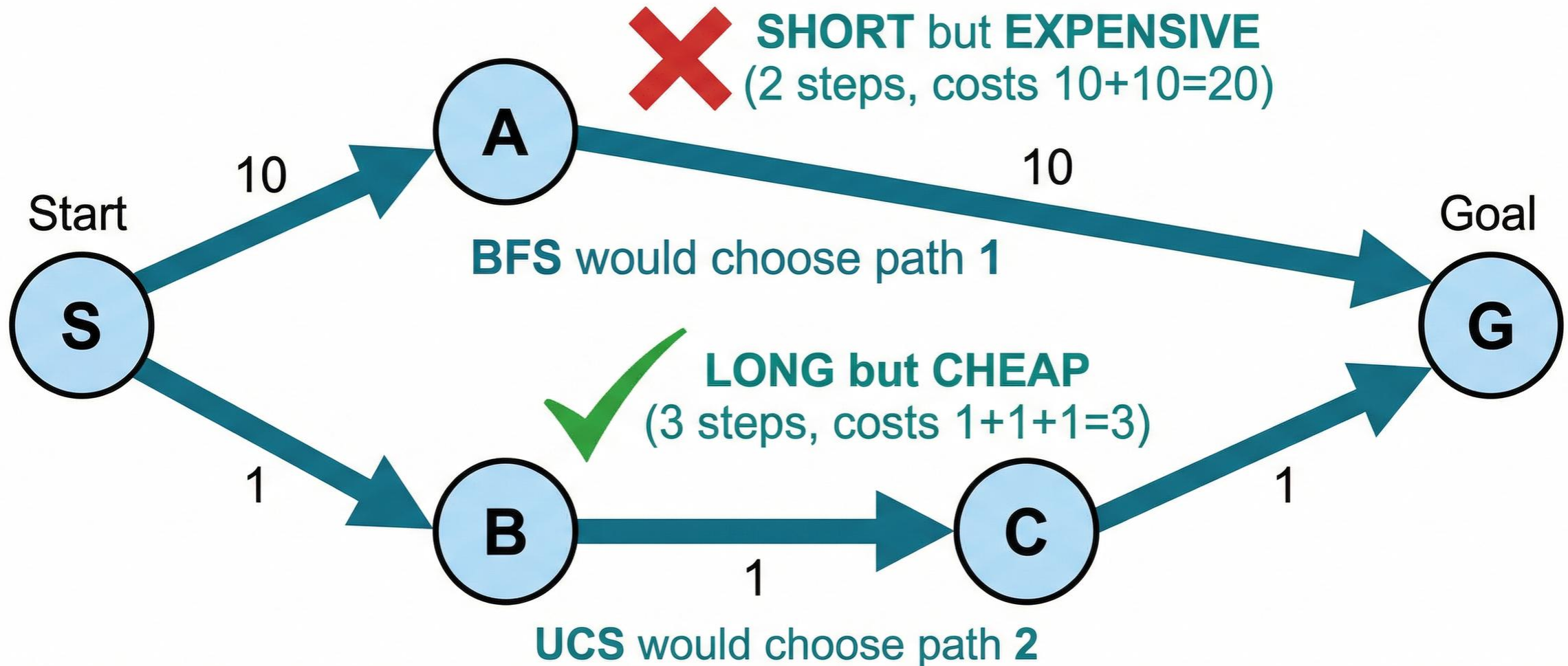
Uniform-Cost Search (UCS)

The cheapest path first.



What if Steps Have Different Costs?

BFS finds **SHORTEST** path (fewest steps).
UCS finds **CHEAPEST** path (lowest total cost).



Uniform-Cost Search: The Core Idea

KEY IDEA: Expand the node with the **LOWEST** path cost $g(n)$ first.

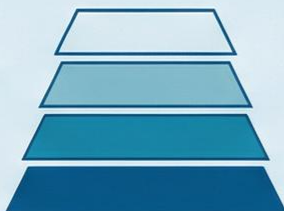
Implementation: Use a **PRIORITY QUEUE** for the frontier.

- Nodes ordered by total path cost from start
- Lowest cost node always expanded first

$g(n)$ = total path cost from start to node n

BFS: Ordered by **DEPTH** (shallow first)

Nodes ordered by
DEPTH (shallowest)
cost is selected
from start



UCS: Ordered by **COST** (cheap first)

Nodes ordered by
COST (cheap first)
and is expanded
at cheap top



Priority Queue

[Arad: 0]
[Arad, $g(n)=0$]

[Zerind: 75]
[Zerind, $g(n)=75$]

[Timisoara: 118]
[Timisoara, $g(n)=118$]

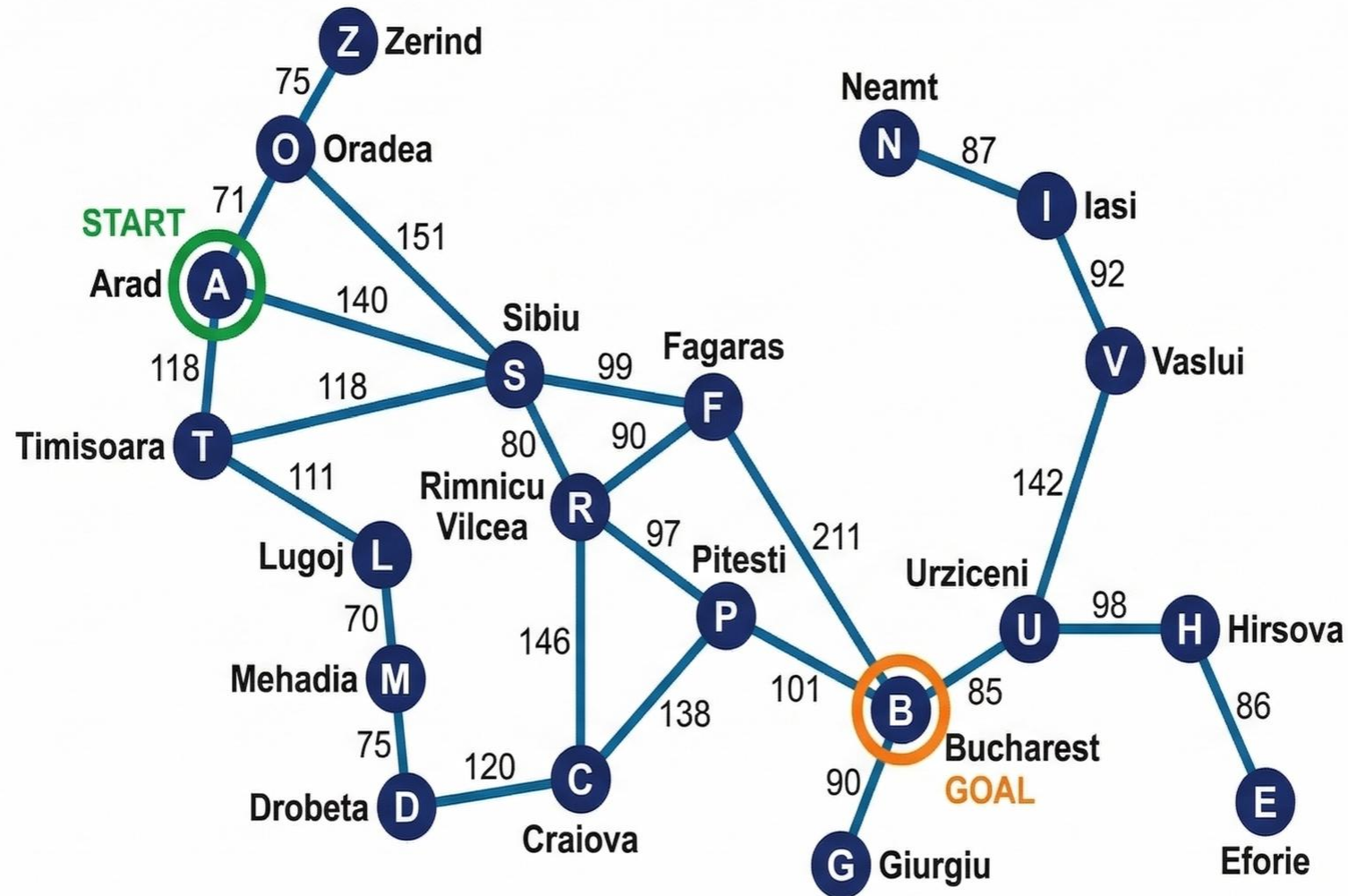
[Sibiu: 140]
[Sibiu, $g(n)=140$]

[Arad: 0]
[Arad, $g(n)=0$]

Lowest cost node
always expanded first
Expand: [Arad: 0]



Example: Finding the Cheapest Route in Romania

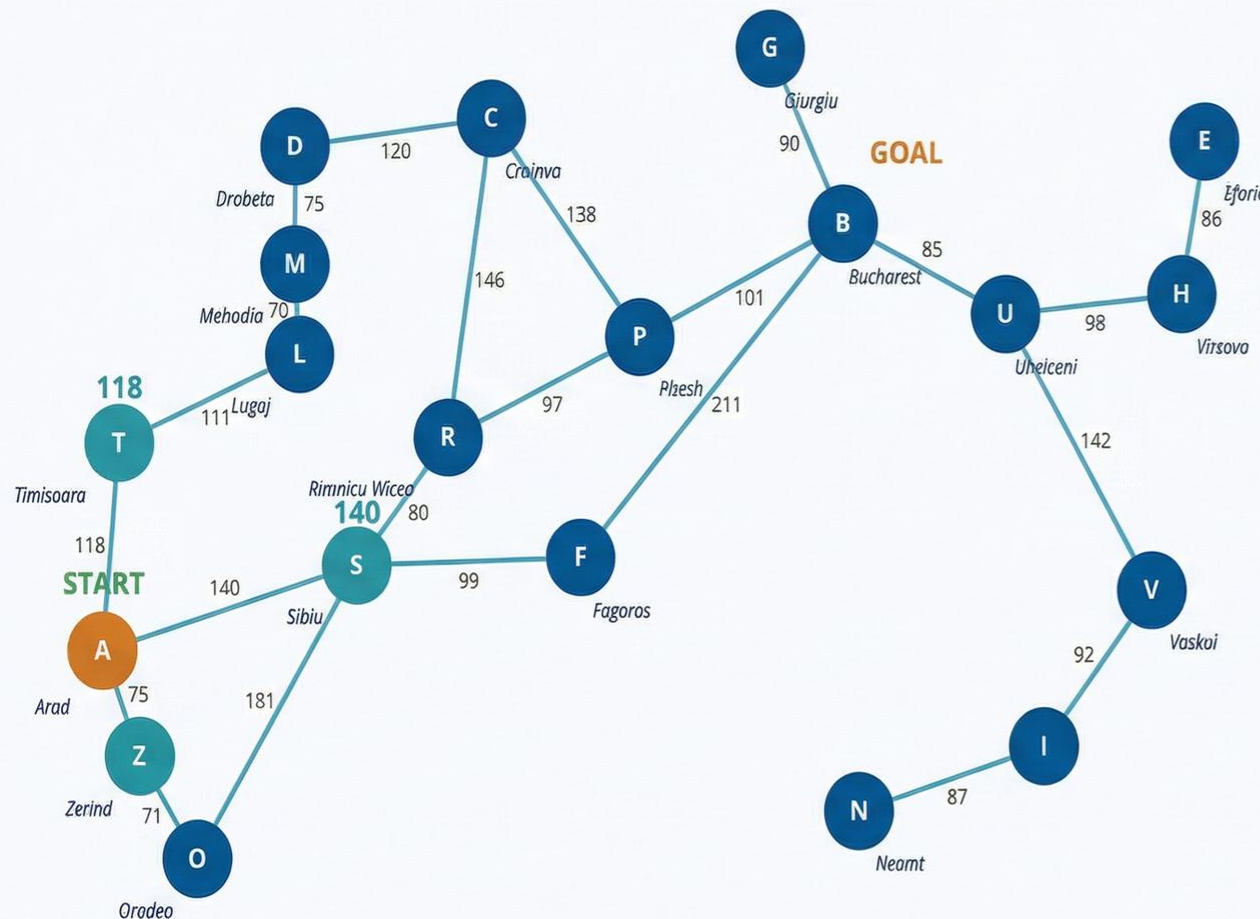


Goal: Find the minimum total distance from Arad to Bucharest.



UCS: Step-by-Step in Romania

Romania Road Map



Priority Queue:
[(Z,75), (T,118), (S,140)]

UCS Romania Demo

Open ucs_romania_demo.html in browser

Step: 2 / 15

Priority Queue:
(Z, cost=75)
(T, cost=118)
(S, cost=140)

[Prev]

[Next]

[Reset]

The LOWEST cost node is ALWAYS expanded next. Watch the priority queue!

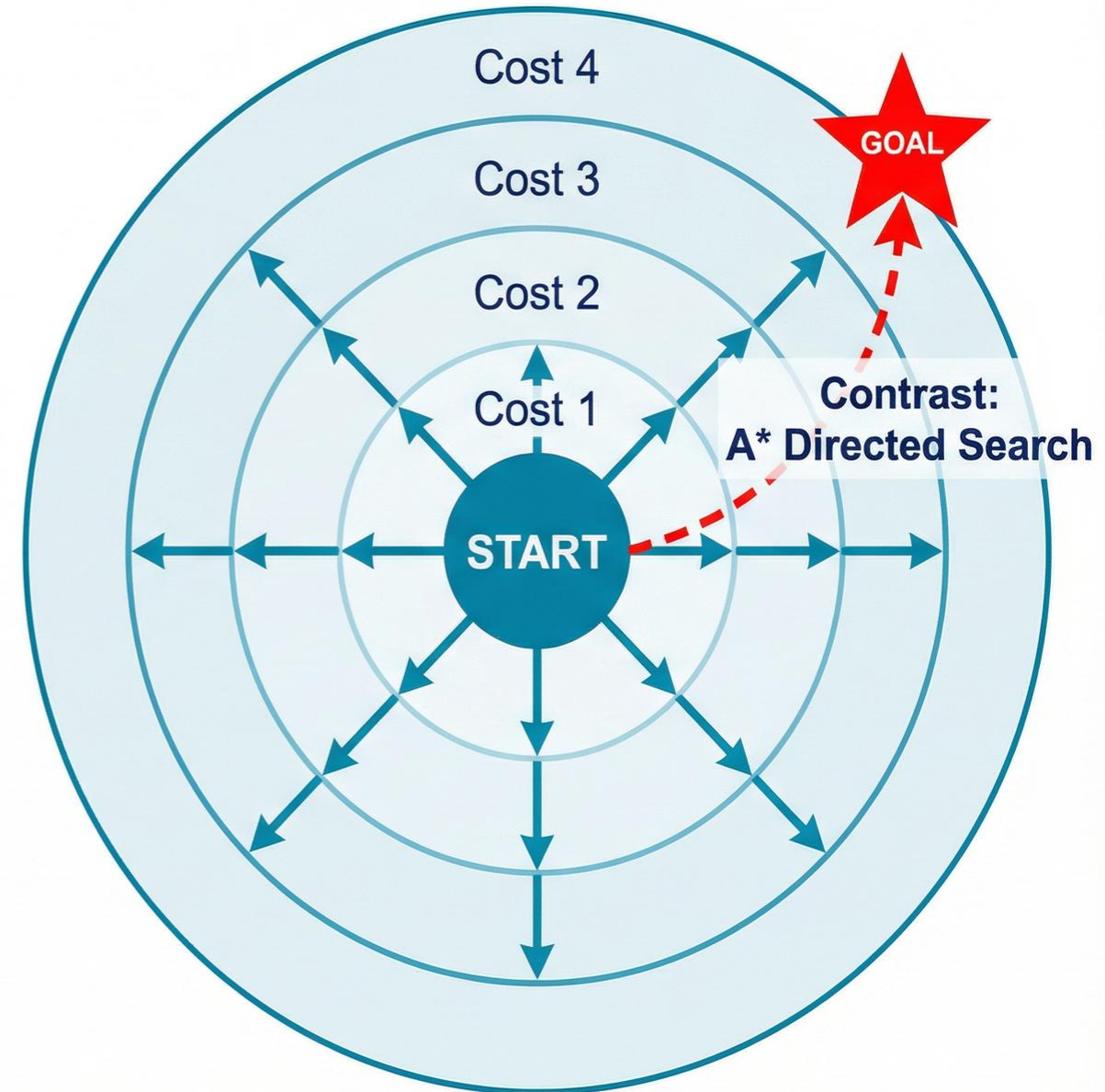


Evaluating Uniform-Cost Search

Criteria	UCS Result	Verdict
Completeness	YES – if step costs are > 0	✓
Optimality	YES – this is its main purpose!	✓ + ★
Time Complexity	$O(b^d)$	!
Space Complexity	$O(b^d)$	!

UCS STRENGTH: OPTIMAL!
Guaranteed to find the cheapest path.

UCS WEAKNESS: Explores in all directions equally. No sense of direction toward goal.

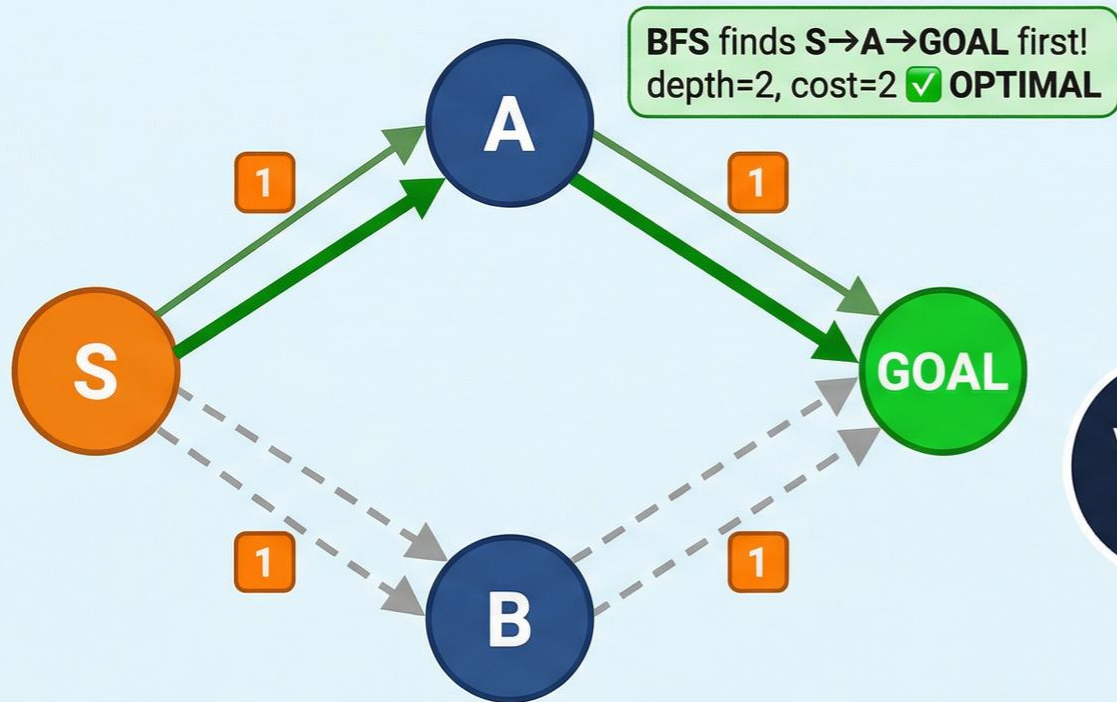




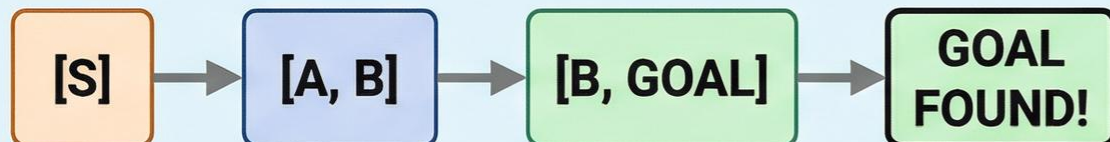
BFS vs UCS: When Costs Matter

Equal Costs → BFS is Optimal

All step costs = 1 | Fewest steps = Lowest total cost



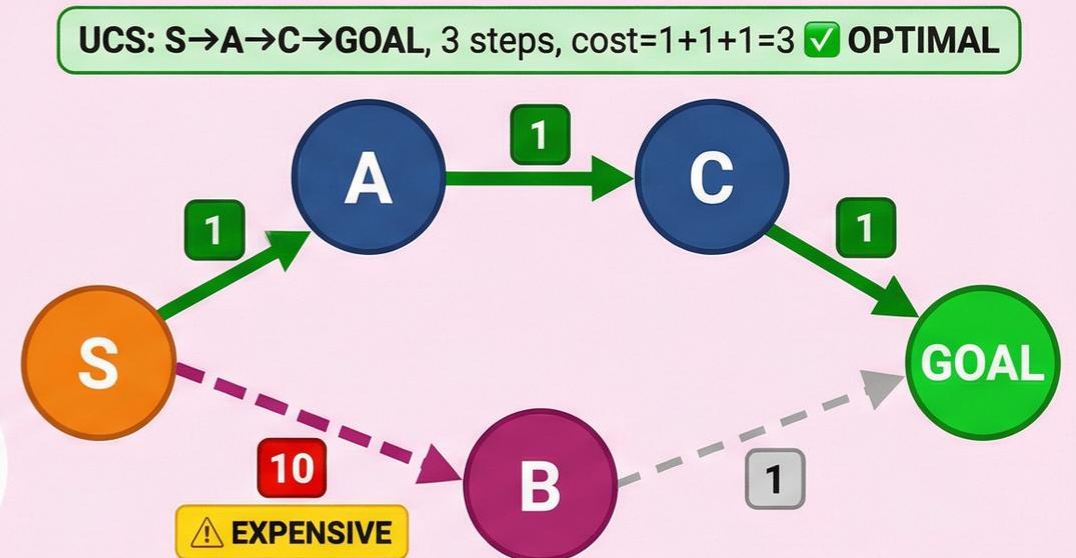
BFS Queue (FIFO)



VS

Unequal Costs → Only UCS is Optimal

BFS ignores costs | UCS always expands the cheapest cumulative cost node



BFS: $S \rightarrow B \rightarrow \text{GOAL}$, 2 steps, cost=10+1=11
✗ NOT OPTIMAL (BFS picks fewest steps, ignores cost)

Algorithm	Path	Steps	Total Cost	Optimal?
BFS	$S \rightarrow B \rightarrow \text{GOAL}$	2 steps	10+1=11	✗ NO
UCS	$S \rightarrow A \rightarrow C \rightarrow \text{GOAL}$	3 steps	1+1+1=3	✓ YES

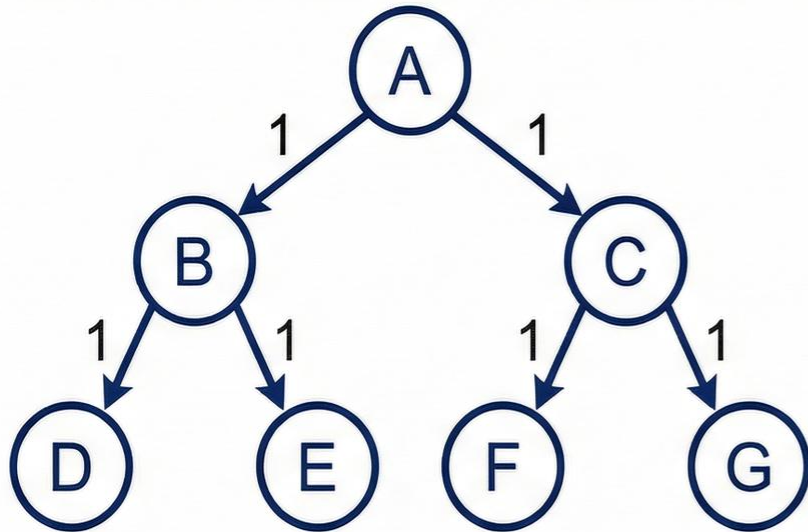
UCS is ALWAYS OPTIMAL

When is UCS the Same as BFS?

The BFS-UCS Relationship

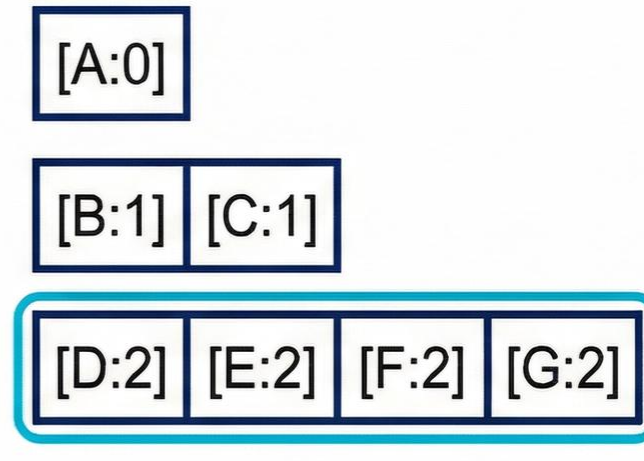
When ALL step costs are EQUAL, UCS behaves exactly like BFS.

BFS Tree



IDENTICAL
BEHAVIOR

UCS Priority Queue



BFS is a special case of UCS where all step costs = 1

This means UCS is the MORE GENERAL algorithm. BFS is just UCS with uniform costs.



Uninformed Search: Summary Table

Algorithm	Complete?	Optimal?	Time	Space
BFS	YES (if cost=1)	YES (if cost=1)	$O(b^d)$	$O(b^d)$
DFS	NO	NO	$O(b^m)$	$O(b^*m)$ BEST
UCS	YES	YES	$O(b^d)$	$O(b^d)$

b=branching factor, d=depth of shallowest solution, m=max depth



What You Should Know Now



BFS vs DFS vs UCS

Know the difference: queue type, expansion order, and when to use each.



Data Structures Matter

Queue (BFS), Stack (DFS), Priority Queue (UCS) - the data structure IS the strategy.



Trade-offs

Completeness vs Optimality vs Time vs Space. No single best algorithm.



Why We Need More

Combinatorial explosion shows why blind search fails for large problems.

You are now ready for INFORMED SEARCH next session!



Next Time: Informed Search

*Searching with a
sense of direction.*

